MemTrust-Lite: A Small Reproducible Study of Provenance and Corroboration in Agent Memory

Abstract

Long-term memory can make language-model agents more useful across sessions, but it also creates a persistent pathway through which untrusted instructions may influence later decisions. This project packages a lightweight synthetic benchmark to compare relevance-only retrieval, provenance filtering, and provenance filtering combined with independent corroboration. The benchmark uses fixed random seeds, three injected-memory settings, explicit metrics, and standard-library Python so that students can reproduce the experiment on a laptop. Because the environment is synthetic and does not invoke an LLM, results should be interpreted as consequences of the simulator's assumptions—not as real-world attack rates or evidence that a deployed agent is secure.

Research question

In a controlled synthetic memory-retrieval environment, how much does a corroboration requirement reduce poisoned-memory selection, and what benign task-utility cost does it introduce compared with relevance-only retrieval and provenance filtering alone?

Motivation and related work

UK university research pages show active work on AI-agent security and trustworthy intelligent systems, including Lancaster University's advertised AI Agent Security PhD positions, Newcastle University's Intelligent Systems group, and the University of Glasgow's 2026 agentic-AI research programme. The literature includes Agent Security Bench and AgentPoison (2024), MemSecBench (2026), and the recent MemForm preprint (October 2026). These sources motivate studying memory as a security surface, but this small package does not claim to reproduce those papers or establish a definitive research gap.

Method

The script creates 2,250 synthetic scenarios across three random seeds, three nominal injected-memory rates (0%, 10%, 25%), and 250 tasks per seed-rate condition. Each scenario contains two clean memory candidates and, with a rate-controlled probability, one injected candidate. Candidate relevance, trust, and corroboration are generated deterministically from fixed seeds. Three policies are compared: (1) relevance-only retrieval; (2) a trust threshold of 0.70; and (3) the same trust threshold plus a minimum of two corroborating sources. The injected instruction is a label only; no malicious action is executed. Each scenario is evaluated under all three policies, yielding 6,750 policy episodes.

Results from the included run

The benchmark was run locally with the included Python script and its five unit tests passed. At the nominal 10% injection setting, attack success among scenarios where a poisoned memory was actually present was **74.1%** for relevance-only retrieval, **8.6%** for provenance filtering, and **0.0%** for provenance plus corroboration. At the nominal 25% setting, the corresponding rates were **70.9%**, **17.1%**, and **0.0%**. On clean-only scenarios, task success was **100.0%** for the first two policies and **91.6%** for provenance plus corroboration at the 0% setting. At 10% injection, overall task success was 92.0%, 99.1%, and 91.6% respectively; at 25%, it was 81.2%, 95.5%, and 92.8%.

These are simulator outputs, not measurements of real LLM agents. The corroboration policy blocks every injected candidate by construction because each injected candidate is assigned only one corroborating source. At the same time, 8% of primary clean memories are assigned only one corroborating source, creating a deliberate utility penalty. The results therefore illustrate a controlled trade-off rather than prove that corroboration solves memory poisoning.

Metrics

Primary security metric: attack success rate among episodes in which a poisoned memory was injected. Utility metrics: expected clean-primary selection on clean-only episodes and overall task success. Secondary metrics include overall poisoned-memory selection, abstention rate, and eligible-candidate count. At the 0% injection setting, attack success is undefined and is reported as NA.

Limitations

This is not an LLM experiment. The trust and corroboration scores are hand-designed, and the defence has an explicit structural advantage because injected entries are assigned only one corroborating source. Results can change if the threat model, score distributions, source-compromise assumptions, or policy thresholds change. The package does not model adaptive attackers, retrieval embeddings, memory writes over time, temporal scope, model reasoning, or end-to-end consequences.

Next steps

A dissertation extension should (a) preregister a threat model, (b) perform threshold sweeps and sensitivity analysis, (c) report confidence intervals across independent seeds, (d) compare against a published benchmark such as ASB or a memory-poisoning benchmark, and (e) evaluate multiple LLMs and memory backends with exact version/configuration records. Any model-based extension should separate deterministic checks from judge-model scoring and disclose cost and failure modes.

Reproducibility and availability

The package uses Python 3.9+ and the standard library only. Inputs and outputs are included in CSV format; figures are generated as SVG. The synthetic dataset is CC BY 4.0, and code is MIT-licensed. See README.md, DATA_LICENSE.md, and references/REFERENCES.md.